Package ‘mlmr’

May 8, 2026

Title Mixed-Effects and Multilevel Models in R

Version 0.1.0

Author Marcus Harris [aut, cre]

Maintainer Marcus Harris <marcus.harris.uconn@gmail.com>

Description Provides a guided Shiny interface and reusable R helpers for specifying, fitting, explaining, and reporting mixed-effects and multilevel models with 'lme4'. The package supports centering decisions, nested and flexible random-effects structures, interaction terms, level-by-level and combined equations, APA-style tables, diagnostics, reproducible code, and software citation support for manuscript workflows.

License MIT + file LICENSE

URL https://marcusharrisphd.com/mlmr/, https://github.com/MarcusHarrisUConn/mlmr

BugReports https://github.com/MarcusHarrisUConn/mlmr/issues

Encoding UTF-8

Roxygen list(markdown = TRUE)

RoxygenNote 7.3.2

Imports shiny,
bslib,
lme4,
ggplot2

Suggests haven,
knitr,
readxl,
rmarkdown

VignetteBuilder knitr

Contents

`mlmr-package` *Mixed-Effects and Multilevel Models in R*

Description

mlmr provides a guided Shiny interface and supporting R toolkit for fitting, understanding, and reporting mixed-effects and multilevel models with **lme4**.

Details

The package is designed for users who want an HLM-style graphical workflow while preserving transparent and reproducible R code. The app supports level-based predictor selection, centering decisions, fixed effects, random intercepts, random slopes, interactions, model diagnostics, APA-style tables, level-by-level equations, combined equations, Tau variance-covariance displays, and reproducible exports.

Launch the app with `run_mlmr()`.

Inspect the current production scope with `mlm_supported_models()`.

To cite the package, use `citation("mlmr")`.

Author(s)

Marcus Harris

See Also

`run_mlmr`, `mlm_supported_models`

`example_hsb` *Generate an HSB-Style Multilevel Example Dataset*

Description

Generates the built-in HSB-style example dataset used by the **mlmr** Shiny app. The data include student-level predictors, school-level predictors, categorical predictors, grouping factors, and outcomes suitable for mixed-effects model demonstrations.

Usage

```
example_hsb(n_schools = 24, min_students = 18, max_students = 34,
  seed = 20260423)
```

Arguments

`n_schools`	Number of schools to simulate.
`min_students`, `max_students`	Minimum and maximum number of students per school.
`seed`	Random seed used to make the generated data reproducible.

Value

A data frame containing simulated student, school, district, and teacher variables.

Examples

```
dat <- example_hsb(n_schools = 4, min_students = 5, max_students = 6)
str(dat)
```

`mlm_apa_tables` *Create Tables and Equations for a Fitted mlmr Model*

Description

Creates manuscript-oriented outputs from a fitted **mlmr** model. These helpers return APA-style tables, level-by-level equations, combined equations, and LaTeX equation displays.

Usage

```
mlm_apa_tables(result, format = c("list", "html", "latex"))

mlm_equations(result)

mlm_latex_equations(result)
```

Arguments

`result`	A fitted result returned by `mlm_fit`.
`format`	Output format for APA tables: `"list"`, `"html"`, or `"latex"`. The list format returns individual data frames; HTML and LaTeX formats return complete manuscript-oriented table documents.

Details

The fixed-effects table includes estimates, standard errors, large-sample test statistics, p values, and Wald 95 percent confidence intervals. For Gaussian mixed models fit with **lme4**, p values are large-sample normal approximations and should be interpreted cautiously.

`mlm_latex_equations()` returns level-by-level equations, a combined full equation, and Tau variance-covariance matrix displays. These outputs are meant to support manuscript preparation and teaching; users should review notation and substantive labels before publication.

Value

`mlm_apa_tables()` returns a named list of tables, an HTML document, or a LaTeX document. `mlm_equations()` returns plain-text equation descriptions. `mlm_latex_equations()` returns raw LaTeX equation components.

Examples

```
if (requireNamespace("lme4", quietly = TRUE)) {
  dat <- example_hsb(n_schools = 4, min_students = 5, max_students = 6)
  spec <- mlm_spec(
    outcome = "mathscore",
    fixed = list(ses = list(center = "CWC")),
    grouping = list(schoolid = "schoolid"),
    random = list(schoolid = list(intercept = TRUE, slopes = character(),
      correlation = TRUE)),
    data = dat
  )
  result <- mlm_fit(spec)
  mlm_apa_tables(result)
  mlm_latex_equations(result)
}
```

`mlm_fit` *Fit a Multilevel Model Specification*

Description

Centers requested predictors, builds an **lme4** formula, and fits the specified mixed-effects or multilevel model.

Usage

```
mlm_fit(spec, REML = TRUE, optimizer = "bobyqa", maxfun = 20000)
```

Arguments

`spec`	An `"mlm_spec"` object created with `mlm_spec`.
`REML`	Logical; whether to use restricted maximum likelihood for Gaussian models. Use `REML = FALSE` when comparing nested models that differ in fixed effects.
`optimizer`	Optimizer passed to **lme4**, such as `"bobyqa"`, `"Nelder_Mead"`, or `"nloptwrap"`.
`maxfun`	Maximum number of function evaluations.

Details

For Gaussian models, `mlm_fit()` calls `lme4::lmer()`. For supported generalized mixed models, it calls `lme4::glmer()` or `lme4::glmer.nb()`. The returned `fit` element is the underlying **lme4** model object, so users can pass it to standard **lme4** methods.

The function performs requested grand-mean and cluster-mean centering before fitting. Users should inspect convergence messages, singular fits, residual patterns, and whether the random-effects structure is supported by the data.

Value

A list containing the fitted model, formula, centered data, centering code, and model specification.

Examples

```
if (requireNamespace("lme4", quietly = TRUE)) {
  dat <- example_hsb(n_schools = 4, min_students = 5, max_students = 6)
  spec <- mlm_spec(
    outcome = "mathscore",
    fixed = list(ses = list(center = "CWC")),
    grouping = list(schoolid = "schoolid"),
    random = list(schoolid = list(intercept = TRUE, slopes = character(),
      correlation = TRUE)),
    data = dat
  )
  fit <- mlm_fit(spec)
  summary(fit$fit)
}
```

`mlm_formula` *Build an lme4 Formula From an mlmr Specification*

Description

Converts an **mlmr** model specification into an **lme4**-compatible model formula, including centered predictor names, interactions, and random effects.

Usage

```
mlm_formula(spec)
```

Arguments

`spec`	An `"mlm_spec"` object created with `mlm_spec`.

Details

The formula uses the effective predictor names created by the centering choices. For example, a predictor with `center = "CWC"` appears with a `_CWC` suffix. Correlated random slopes are represented with `(1 + slope | group)` syntax, whereas independent random slopes are represented with separate `(1 | group)` and `(0 + slope | group)` terms.

Value

A model formula.

Examples

```
dat <- example_hsb(n_schools = 4, min_students = 5, max_students = 6)
spec <- mlm_spec(
  outcome = "mathscore",
  fixed = list(ses = list(center = "CWC")),
  grouping = list(schoolid = "schoolid"),
  random = list(schoolid = list(intercept = TRUE, slopes = "ses",
    correlation = TRUE)),
  data = dat
```

```
)
mlm_formula(spec)
```

`mlm_software_table` *Report R and Package Versions*

Description

Creates a reproducibility-oriented table and APA-style prose statement reporting the R version and package versions used in an analysis.

Usage

```
mlm_software_table(packages = mlm_default_packages())

mlm_software_apa(packages = mlm_default_packages())

mlm_papaja_code(packages = mlm_default_packages(), bib_file = "r-references.bib")
```

Arguments

packages	Character vector of software or R package names to report. Use "R" to include the R version.
bib_file	Name of the BibTeX file to create or update when using the optional **papaja** workflow.

Details

mlm_papaja_code() returns R code that users can run when **papaja** is installed. The generated code uses papaja::r_refs() to create package references and papaja::cite_r() to create an APA-style R/package citation statement. **papaja** is optional and is not required to install or run **mlmr**.

Value

mlm_software_table() returns a data frame. mlm_software_apa() returns a manuscript-ready prose statement. mlm_papaja_code() returns character lines of reproducible R code.

Examples

```
mlm_software_table()
mlm_software_apa()
cat(mlm_papaja_code(), sep = "\n")
```

mlm_spec *Create a Multilevel Model Specification*

Description

Creates a structured model specification used by mlm_formula, mlm_fit, the Shiny app, and reporting helpers.

Usage

```
mlm_spec(outcome, distribution = "gaussian", link = "identity",
  fixed = list(), grouping = list(), nesting = NULL, structure = "nested",
  random = list(), interactions = list(), predictor_levels = list(),
  data = NULL)
```

Arguments

outcome	Character string giving the name of the outcome variable in data.
distribution	Outcome distribution. The primary production workflow uses "gaussian". Advanced workflows may use "binomial", "poisson", "negative binomial", or "Gamma".
link	Link function for generalized mixed models. For Gaussian models the default "identity" link is used.
fixed	Named list of fixed predictors. Each entry may include a center value: "none", "GMC" for grand-mean centering, or "CWC" for cluster-mean centering.
grouping	Named list mapping conceptual grouping names to data columns. For example, list(schoolid = "schoolid") or list(schoolid = "schoolid", districtid = "districtid").
nesting	Optional text description of the nesting or design structure.
structure	Model structure label, such as "nested" or "crossed". This label is used for documentation and guidance; the fitted model is determined by the grouping and random-effect terms.
random	Named list describing random intercepts, slopes, and correlation structure for each grouping factor. Each entry may contain intercept = TRUE, slopes = c(...), and correlation = TRUE or FALSE.
interactions	List of character vectors describing interaction terms, for example list(c("ses", "sector")).
predictor_levels	Optional named list declaring conceptual predictor levels, such as list(level1 = c("ses"), level2 = c("meanses")). This is used for app display and reporting.
data	Data frame used to fit the model.

Details

mlm_spec() records the modeling choices made in the app or in code. It does not fit the model by itself. Centered variables are created during mlm_fit and are reflected in the generated **lme4** formula.

Users are responsible for confirming that grouping IDs, predictor levels, centering decisions, interactions, and random effects match the research design.

Value

An object of class "mlm_spec", represented as a structured list. It can be passed to mlm_formula or mlm_fit.

Examples

```
dat <- example_hsb(n_schools = 4, min_students = 5, max_students = 6)
spec <- mlm_spec(
  outcome = "mathscore",
  fixed = list(
    ses = list(center = "CWC"),
    meanses = list(center = "GMC"),
    sector = list(center = "none")
  ),
  grouping = list(schoolid = "schoolid"),
  random = list(schoolid = list(intercept = TRUE, slopes = "ses",
    correlation = TRUE)),
  interactions = list(c("ses", "sector")),
  predictor_levels = list(level1 = "ses", level2 = c("meanses", "sector")),
  data = dat
)
mlm_formula(spec)
```

---

mlm_supported_models *Summarize the Current mlmr Modeling Scope*

Description

Returns a table describing the modeling, reporting, import, diagnostic, and export features that are currently supported, experimental, or planned in **mlmr**.

Usage

```
mlm_supported_models()
```

Details

This helper is intended for transparent documentation and teaching. It gives users a compact way to see the current production path, advanced workflows, and areas that require extra statistical judgment or future development.

Value

A data frame with columns Area, Status, Scope, and User_responsibility.

Examples

```
mlm_supported_models()
```

run_mlmr *Launch the mlmr Shiny App*

Description

Launches the **mlmr** Shiny interface for mixed-effects and multilevel modeling with **lme4**.

Usage

```
run_mlmr(launch.browser = TRUE, port = getOption("shiny.port", NULL))
```

Arguments

`launch.browser`	Logical; whether to open the app in the default browser.
`port`	Optional port passed to `shiny::runApp`.

Details

The app opens with a built-in HSB-style example dataset and a preset model so users can test the complete workflow before uploading their own data. Uploaded data can be provided as CSV, TSV/TXT, Excel, SPSS, SAS, or Stata files when the optional import packages are installed.

The app is intended to guide model specification, estimation, diagnostics, equation display, APA-style tables, and reproducible exports. The strongest current production path is Gaussian two-level and three-level nested mixed-effects models. Use `mlm_supported_models()` to inspect the current supported, experimental, and planned scope.

Value

Runs the Shiny application. This function is called for its side effect and does not return a modeling object.

See Also

`mlm_supported_models`, `mlm_spec`, `mlm_fit`

Examples

```
if (interactive()) {
  run_mlmr()
}
```

Index